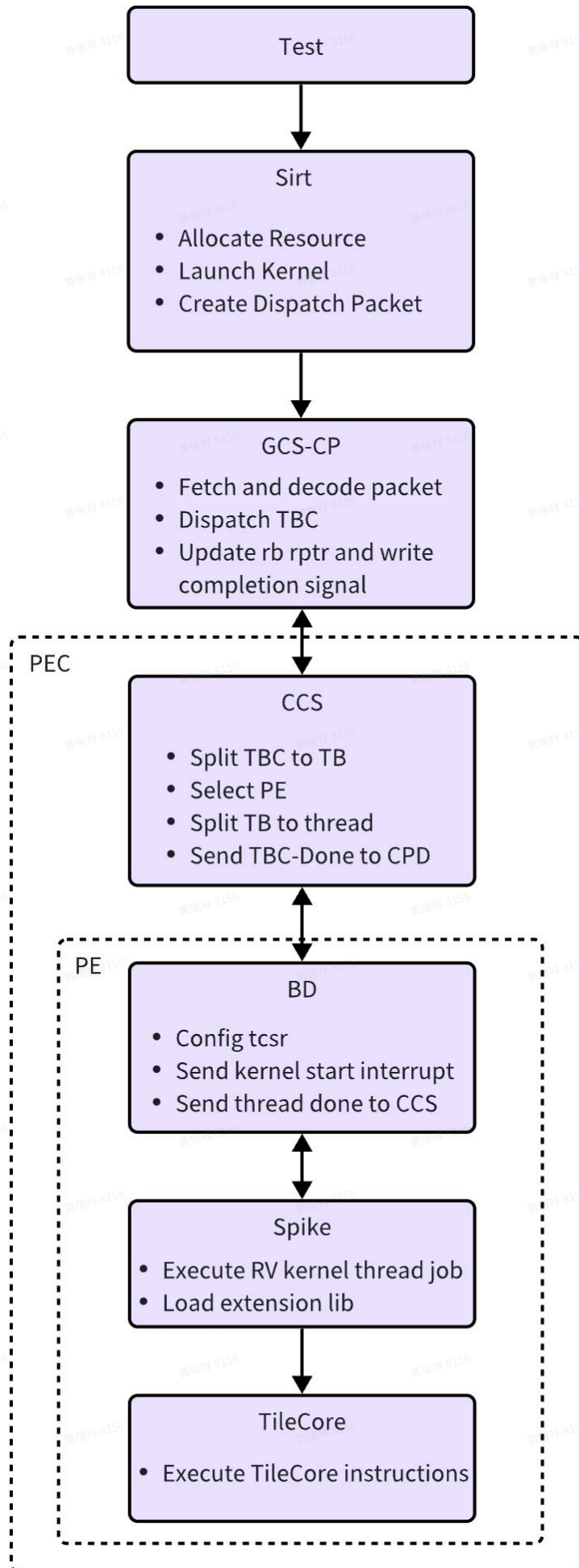


Arch Model概述

1. Overview

Arch model is mainly used to verify ISA and software high-level features.



2. Functions

2.1 Test

- Asm tests for tilecore instructions per module
- High-level tests for system and tilecore

2.2 Sirt

- Driver
- Allocate resource
- Launch kernel
- Create dispatch packet and write to ring buffer

2.3 GCS-CP

- CPF fetch packet in ring buffer
- CPE decode packet
- CPD dispatch Thread-Block-Cluster job to CCS
- Update ring buffer read pointer
- Write completion signal when job done

2.4 CCS

- Receive TBC from CPD and split TBC to TB
- Select PE referred to RV, tilecore, L2B resources in PEC
- Split TB to thread and send to BD
- Send TBC done signal to CPD

2.5 BD

- Config tcsr
- Send kernel start interrupt to RV
- Send Thread done signal to CCS

2.6 Spike

- Execute RV thread
- Load extension library

2.7 TileCore

- Register extension lib to spike named "tilecore"
- Using "tcore_decode" function to get processor info, instruction and pc from spike
- Decode instruction's tuop to execute tilecore module

2.7.1 Tile Front-End

- No tile front-end in arch model, and decode instructions in each tilecore module

2.7.2 Tile LSU

- TLSU is the Load/Store module in tilecore, supports load, store and atomic functions
- The code entry is "tcore_exec_mem", the function is in tlsu.cc
- Decode mem_fmt and mem_type from memuop to execute normal load/store, block load/store and atomic

2.7.3 Tile MAC Unit

- TMAC is the matrix calculation unit module in tilecore
- The code entry is "tcore_exec_mma", the function is in tilecore_exec_mma.cc
- Decode src/dst fmt from 38 mmauops to execute M32/M16/M8 normal/A8W4/A6W6 micro operations

2.7.4 Tile ALU

- TALU is the vector-level arithmetic logic unit in tilecore
- Supported data format: FP32 & FP16 & BF16
- Supported operation: Conversion, Transpose, add, mul, min, max, fma
- The code entry is "tcore_exec_alu", the function is in tilecore_exex_alu.cc

2.7.5 Tile Move

- TMOV is responsible for data transfer between the tile registers and the RV registers
- Supported transfer: Tile to Vector, Vector to tile, Tile to Scaler, Scaler to Tile, Immediate to Tile
- The code entry is "tcore_exec_mv", the function is in tilecore_exex_mv.cc

2.7.6 Tile SFU

- TSFU is a vector-level operation unit in TileCore, mainly completes the calculation special function instructions.

- Supported: $\sin(2\pi * x)$, $\cos(2\pi * x)$, $\text{sigmoid}(x)$, $\tanh(x)$, $\text{rcp}(x)$, $\text{sqrt}(x)$, $\text{rsqrt}(x)$, $\log_2(x)$, $\exp_2(x)$.
- Supported data format: FP32 & FP16 & BF16

2.7.7 Tile DTE

- TDTE is used to copy large chunks of data
- The entry is "tcore_exec_memacp", the function is in tdte.cc
- Decode acpuop to execute linear copy, tiled copy and tiled-linear-conversion

2.7.8 Tile Sync

- **TODO**
- TSYNC is responsible for synchronization between multiple threads.
- The code entry is "tcore_exec_sync", the function is in tilecore.cc

2.7.9 Tile Config

- The TCSR module is responsible for reading and writing CSR registers
- Supported operation: TCSR Read, TCSR Write, TCSR Write Immediate
- The code entry is "tcore_exec_cfg", the function is in tilecore.cc

3. Library

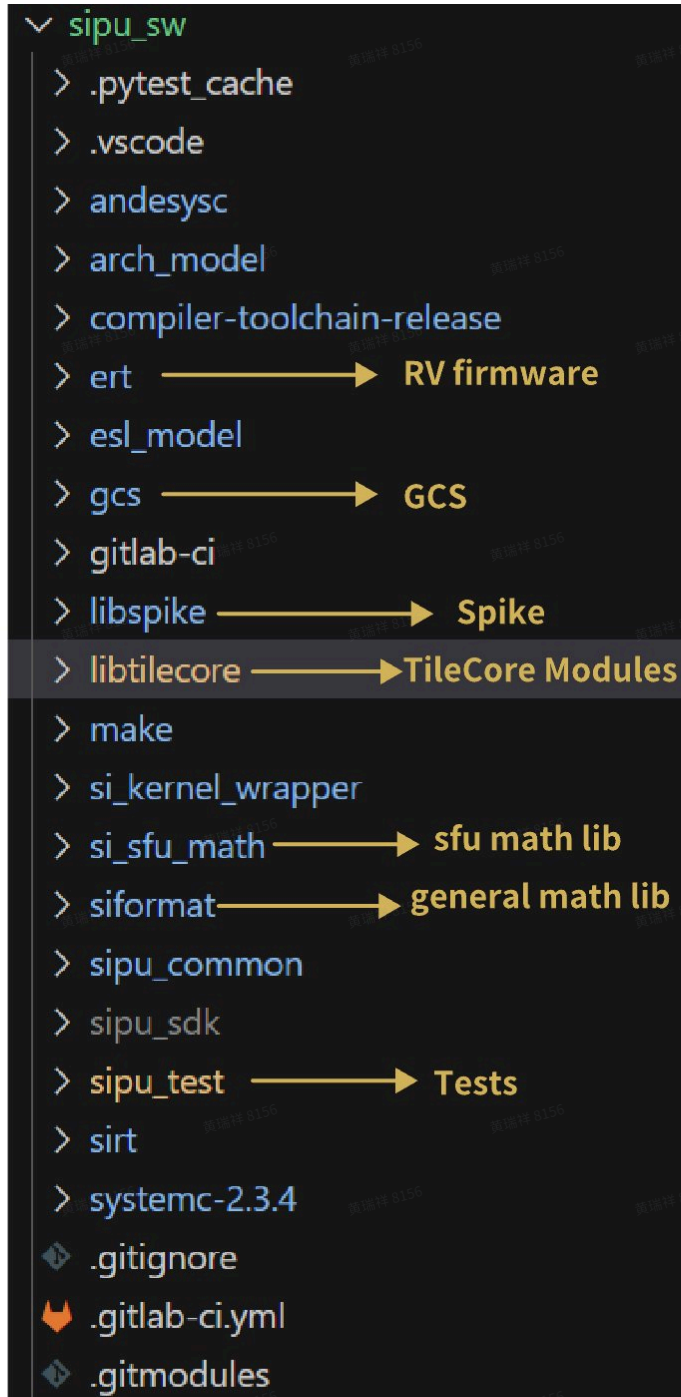
3.1 siformat

- SiFormat Library is the computational functional module for tilecore with bit accurate target
- SiFormat is introduced into the TileCore project as a library
- The SiFormat library consists of four modules: base, SIMD, Vector, and Matrix, which support scale, conversion, vector, and matrix calculations respectively

3.2 si_sfu_math

- SI SFU MATH library is special function module for tilecore with Bit Accurate target
- si_sfu_math is used for tilecore (DV and arch model)

4. Code Structure



5. Environment

Please refer to [国 SiPU架构&软件协同开发环境](#) to setup the environment, build and run.

6. Debug

6.1 vscode

- launch.json: replace "program" with your program location.

```
// Use IntelliSense to learn about possible attributes.
// Hover to view descriptions of existing attributes.
// For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
"version": "0.2.0",
"configurations": [
  {
    "name": "(gdb) Launch",
    "type": "cppdbg",
    "request": "launch",
    "program": "/data/users/wangchenghui/work/proj/sipu_sw/sipu_test/build/isa/tlsu/tlsu_tile_atomic_min",
    "args": [],
    "stopAtEntry": false,
    "cwd": "${workspaceFolder}",
    "environment": [],
    "externalConsole": false,
    "MIMode": "gdb",
    "miDebuggerPath": "/usr/bin/gdb",
    "setupCommands": [
      {
        "description": "Enable pretty-printing for gdb",
        "text": "-enable-pretty-printing",
        "ignoreFailures": true
      }
    ]
  }
]
```

6.2 gdb

代码块

```
1  # tlsu example
2  cd ~/work/proj/sipu_sw/sipu_test/build/isa/tlsu
3  gdb ./tlsu_tile_atomic_min
```